Grado en Ingeniería Informática
Algorítmica - Prácticas

Cristina Zuheros Montes

Departamento de Ciencias de la Computación
Universidad de Granada

Marzo, 2026



UNIVERSIDAD
DE GRANADA

Este documento está protegido por la Ley de Propiedad Intelectual (Real Decreto Legislativo 1/1996, de 12 de abril).

Queda expresamente prohibido su uso o distribución sin autorización del autor.

Práctica 3

Algoritmos Greedy.

Algoritmos Greedy

- Algoritmo para resolver problemas que consiste en tomar siempre la mejor opción posible en cada paso en base a criterios locales.

- Intuición: "Elegir siempre lo mejor en este momento esperando que conduzca a una solución óptima global."



UNIVERSIDAD DE GRANADA

Elementos de un algoritmo Greedy

- Conjunto de Candidatos (C): elementos disponibles para resolver el problema.
- Conjunto de Seleccionados (S): representa al conjunto de decisiones tomadas hasta este momento.
- Función Selección: elige, de entre los candidatos restantes en C, cuál es el mejor candidato actual según un criterio local.
- Función de Factibilidad: antes de añadir el candidato elegido a la solución S, esta función comprueba si es posible o válido incluirlo.
- Función Solución: función que evalúa si el conjunto S ya es una solución del problema.
- Función Objetivo: función que da valor a la solución alcanzada. Aunque el algoritmo elige candidatos basándose en la función de selección, el éxito total se mide con esta función.

Esquema general

```
Greedy(C:conjunto de cantidatos):  conjunto solución
     S ← ∅
     while C ≠ ∅ y !Solución(S) do
          x ← Selección(C)
          C ← C - {x}
          if factible(S ∪ {x}) then
               S ← S ∪ {x}
          end if
     end while
     if Solución(S) then
          return S
     else
          return ‘‘No hay solución’’
     end if
```

Características

- No son previsores.
- No tiene en cuenta las decisiones pasadas.
- No reconsidera decisiones (no backtracking).
- Suelen ser rápidos y fáciles de implementar.
- Pueden no encontrar solución, aunque exista.
- No siempre garantizan la solución óptima.



UNIVERSIDAD DE GRANADA

Aplicación

- Se aplican típicamente a problemas de optimización (máximos o mínimos).

- Es apropiado cuando el problema tiene:
  - Propiedad de elección voraz: existe una solución óptima que comienza con la elección realizada por el algoritmo.
  - Subestructura óptima: una solución óptima contiene soluciones óptimas a sus subproblemas.

Ejemplo: Selección de actividades
Dado un conjunto de actividades con tiempos de inicio y fin, queremos seleccionar el máximo número de actividades compatibles.



UNIVERSIDAD DE GRANADA

Solución óptima

Una solución Greedy NO garantiza la solución óptima de un problema...

Cuando no está garantizado que la mejor decisión local lleve a la solución global óptima.

1. Cuando no se cumple la propiedad de elección voraz: incluso el primer paso puede impedir alcanzar la mejor solución global.

Ejemplo Caso 1: Elección voraz

Se dispone de monedas de 1, 3 y 4. Hay que elegir las monedas para que, en combinación, den un valor de 6 a la vez que se minimiza el número de monedas utilizadas.

- Greedy: Elige la moneda de mayor valor que sea menor a la cantidad restante en cada paso.
  - Paso 1: Elige la moneda de 4 ($4 \leq 6$). Cantidad que queda: 2
  - Paso 2: Elige la moneda de 1 ($1 \leq 2$). Cantidad que queda: 1
  - Paso 3: Elige la moneda de 1 ($1 \leq 1$)
- Greedy: $4 + 1 + 1 \simeq 3$ monedas
- Óptimo: $3 + 3 \simeq 2$ monedas

UNIVERSIDAD DE GRANADA

Solución optima

Una solución Greedy NO garantiza la solución óptima de un problema...

Cuando no está garantizado que la mejor decisión local lleve a la solución global óptima.

1. Cuando no se cumple la propiedad de elección voraz: incluso el primer paso puede impedir alcanzar la mejor solución global.

2. El problema no tiene subestructura óptima.



UNIVERSIDAD DE GRANADA

Ejemplo Caso 2: Sin subestructura óptima

Se desea viajar de Granada a París. Hay dos rutas posibles, pasando por Barcelona o por Lyon.

- Granada → BCN (tren) = 20
- BCN → París (avión) = 10
- Granada → Lyon (tren) = 50
- Lyon → París (tren) = 60

Existe una penalización de 100 si se cambia de medio de transporte.

- Subproblema: Mejor ruta local BCN → París = 10
- Coste: Granada → BCN → París = $20 + 10 + 100 = 130$
- Óptimo: Granada → Lyon → París = $50 + 60 = 110$
- La mejor solución del subproblema no forma parte de la solución óptima global



UNIVERSIDAD DE GRANADA

Solución optima

Una solución Greedy NO garantiza la solución óptima de un problema...

Cuando no está garantizado que la mejor decisión local lleve a la solución global óptima.

1. Cuando no se cumple la propiedad de elección voraz: incluso el primer paso puede impedir alcanzar la mejor solución global.
2. El problema no tiene subestructura óptima.
3. Cuando la función de selección depende demasiado del orden.
4. ...



UNIVERSIDAD DE GRANADA

Ejemplo Caso 3: Dependencia

Hay que asignar 2 trabajadores a 2 tareas, sabiendo que cada trabajador tarda un tiempo distinto en cada tarea.

	Tarea 1	Tarea 2
Trabajador A	3	4
Trabajador B	2	10

- Greedy (dependiente del orden): Asignar a cada trabajador la tarea disponible para la que tarda menos.
  - Paso 1: $A \rightarrow$ Tarea 1, porque tarda 3.
  - Paso 2: $B \rightarrow$ Tarea 2, que tarda 10.
- Greedy: $3 + 10 = 13$
- Si los trabajadores estuvieran en orden inverso $B \rightarrow$ Tarea 1 y $A \rightarrow$ Tarea 2, se obtiene $2 + 4 = 6$

UNIVERSIDAD DE GRANADA

Ejemplo

El problema del cambio de monedas.

Elementos del algoritmo Greedy

Dado un conjunto de monedas, de cantidad ilimitada, y una cantidad P se debe devolver esa cantidad usando el menor número posible de monedas.



UNIVERSIDAD DE GRANADA

Elementos del algoritmo Greedy

Dado un conjunto de monedas, de cantidad ilimitada, y una cantidad P se debe devolver esa cantidad usando el menor número posible de monedas.

- **Conjunto de candidatos** C**:** Monedas disponibles de cada tipo.



UNIVERSIDAD
DE GRANADA

Elementos del algoritmo Greedy

Dado un conjunto de monedas, de cantidad ilimitada, y una cantidad P se debe devolver esa cantidad usando el menor número posible de monedas.

- **Conjunto de candidatos** C**:** Monedas disponibles de cada tipo.
- **Solución parcial** S**:** Monedas ya seleccionadas.



UNIVERSIDAD DE GRANADA

Elementos del algoritmo Greedy

Dado un conjunto de monedas, de cantidad ilimitada, y una cantidad P se debe devolver esa cantidad usando el menor número posible de monedas.

- **Conjunto de candidatos** C**:** Monedas disponibles de cada tipo.
- **Solución parcial** S**:** Monedas ya seleccionadas.
- **Función de solución:** Conjunto de monedas que sumen P.



UNIVERSIDAD DE GRANADA

Elementos del algoritmo Greedy

Dado un conjunto de monedas, de cantidad ilimitada, y una cantidad P se debe devolver esa cantidad usando el menor número posible de monedas.

- **Conjunto de candidatos** C**:** Monedas disponibles de cada tipo.
- **Solución parcial** S**:** Monedas ya seleccionadas.
- **Función de solución:** Conjunto de monedas que sumen P.
- **Función de selección:** Escoger la moneda de mayor valor entre las disponibles.



UNIVERSIDAD DE GRANADA

Elementos del algoritmo Greedy

Dado un conjunto de monedas, de cantidad ilimitada, y una cantidad P se debe devolver esa cantidad usando el menor número posible de monedas.

- **Conjunto de candidatos** C**:** Monedas disponibles de cada tipo.
- **Solución parcial** S**:** Monedas ya seleccionadas.
- **Función de solución:** Conjunto de monedas que sumen P.
- **Función de selección:** Escoger la moneda de mayor valor entre las disponibles.
- **Función de factibilidad:** La suma de las monedas seleccionadas no supera P.



UNIVERSIDAD DE GRANADA

Elementos del algoritmo Greedy

Dado un conjunto de monedas, de cantidad ilimitada, y una cantidad P se debe devolver esa cantidad usando el menor número posible de monedas.

- **Conjunto de candidatos** C**:** Monedas disponibles de cada tipo.
- **Solución parcial** S**:** Monedas ya seleccionadas.
- **Función de solución:** Conjunto de monedas que sumen P.
- **Función de selección:** Escoger la moneda de mayor valor entre las disponibles.
- **Función de factibilidad:** La suma de las monedas seleccionadas no supera P.
- **Función objetivo:** Minimizar el número de monedas.



UNIVERSIDAD DE GRANADA

Solución Greedy

Dado un conjunto de monedas, de cantidad ilimitada, y una cantidad P se debe devolver esa cantidad usando el menor número posible de monedas.

Ejemplo: Se dispone de monedas de 1,2,5,10, 20 y 50 céntimos de euro. Si $P = 0{,}89$ euros, el conjunto de monedas devuelto sería:

Moneda	**Número de monedas**	**Cantidad restante**
50 cent	1	0.39
20 cent	1	0.19
10 cent	1	0.09
5 cent	1	0.04
2 cent	2	0

Solución: 50c, 20c, 10c, 5c, 2c, 2c $\simeq$ 6 monedas



UNIVERSIDAD DE GRANADA

Implementación - I

```
0  vector<Moneda> greedy(const vector<int>& candidatos, int cantidad) {
1      vector<Moneda> solucion = {};
2
3      int restante = cantidad;
4
5      int i = 0;
6      while (i < candidatos.size() && !esSolucion(restante)) {
7          int mTipo = seleccionar(candidatos, i++);
8
9          if (factible(mTipo, restante)) {
10             int mCantidad = restante / mTipo;
11
12             if (mCantidad > 0) {
13                 solucion.push_back({mTipo, mCantidad});
14                 restante -= mCantidad * mTipo;
15             }
16         }
17     }
18
19     return solucion;
20 }
```

En lugar de eliminar las monedas candidatas, se ha optado por recorrerlas.

Implementación - II

```
0
1  struct Moneda {
2      int valor;
3      int cantidad;
4  };
5
6  bool esSolucion(int restante) { O(1)
7      return restante == 0;
8  }
9
10 bool factible(int valorMoneda, int restante) { O(1)
11     return valorMoneda <= restante;
12 }
13
14 int seleccionar(const vector<int>& monedas, int i) { O(1)
15     if (i < monedas.size()) {
16         return monedas[i]; // asumo orden decreciente
17     }
18     return -1;
19 }
```



UNIVERSIDAD DE GRANADA

Eficiencia teórica

```
0  vector<Moneda> greedy(const vector<int>& candidatos, int cantidad) {
1      vector<Moneda> solucion = {};
2
3      int restante = cantidad;
4
5      int i = 0;
6      while (i < candidatos.size() && !esSolucion(restante)) {
7          int mTipo = seleccionar(candidatos, i++);
8
9          if (factible(mTipo, restante)) {
10             int mCantidad = restante / mTipo;
11
12             if (mCantidad > 0) {
13                 solucion.push_back({mTipo, mCantidad});
14                 restante -= mCantidad * mTipo;
15             }
16         }
17     }
18
19     return solucion;
20 }
```

Sea n el número de tipos de moneda. El algoritmo recorre el vector de candidatos una única vez. En cada iteración realiza únicamente operaciones de coste constante $\rightarrow O(n)$.



UNIVERSIDAD DE GRANADA

¿Solución óptima? Intuición

El algoritmo greedy elige en cada paso la moneda de mayor valor posible. ¿Siempre produce la solución óptima?

- Para el sistema de monedas $\{1c, 2c, 5c, 10c, 20c, 50c\}$.
  Cada moneda es múltiplo o una combinación eficiente de monedas menores. Para cada moneda, cualquier forma de representarla con monedas más pequeñas requiere igual o mayor número de monedas.
  Ejemplo: $50cnt = 20c + 20c + 10c$ (3 monedas) $\Longrightarrow$ peor que usar 1 moneda

- El algoritmo greedy siempre encuentra una solución óptima.

- Esto no es cierto para cualquier conjunto de monedas, depende del sistema de monedas. Sea $\{1, 3, 4\}$ y $P = 6$
  - Greedy: $4 + 1 + 1 = 3$ monedas
  - Óptimo: $3 + 3 = 2$ monedas



UNIVERSIDAD DE GRANADA

¿Solución óptima? Demostración

El algoritmo greedy elige en cada paso la moneda de mayor valor posible. Demostramos que para el sistema de monedas $\{1c, 2c, 5c, 10c, 20c, 50c\}$ la solución es óptima.



UNIVERSIDAD DE GRANADA

¿Solución óptima? Demostración - I

El algoritmo greedy elige en cada paso la moneda de mayor valor posible. Demostramos que para el sistema de monedas $\{1c, 2c, 5c, 10c, 20c, 50c\}$ la solución es óptima.

Viendo el sistema monetario, podemos deducir fácilmente que en una solución óptima hay como máximo:

- 1 moneda de $1c$
- 2 monedas de $2c$
- 1 moneda de $5c$
- 1 moneda de $10c$
- 2 monedas de $20c$.



UNIVERSIDAD DE GRANADA

¿Solución óptima? Demostración - II

Sea P una cantidad. Escribimos:

$$P = 50a + r, \quad 0 \leq r < 50$$

En una solución óptima tiene que haber $a = \left\lfloor \frac{P}{50} \right\rfloor$ monedas de 50 ya que una moneda de 50 sustituida por menores requiere al menos 3 monedas. Por tanto, usar una moneda de $50c$ nunca es peor que reemplazarla por monedas menores.

Aplicamos el mismo razonamiento con el resto:

$$r = 20b + s, \quad 0 \leq s < 20$$

Sabemos que no puede haber 3 monedas de 20. La elección $b = \left\lfloor \frac{r}{20} \right\rfloor$ lo verifica ya que $r < 50$ y además sabemos que si no tomamos una moneda de $20c$ pudiendo hacerlo, habría que sustituirla por al menos 2 monedas más pequeñas. Por tanto, usar una moneda de $20c$ nunca es peor que reemplazarla por monedas menores.

¿Solución óptima? Demostración - III

Sea el resto

$$s = 10c + t, \quad 0 \leq t < 10$$

Sabemos que no puede haber 2 monedas de 10. La elección $c = \left\lfloor \frac{s}{10} \right\rfloor$ lo verifica ya que $s < 20$ y usar una moneda de 10c nunca es peor que reemplazarla por monedas menores. Luego:

$$t = 5d + u, \quad 0 \leq u < 5$$

Sabemos que no puede haber 2 monedas de 5. La elección $d = \left\lfloor \frac{t}{5} \right\rfloor$ lo verifica ya que $t < 10$ y usar una moneda de 5c nunca es peor que reemplazarla por monedas menores. Finalmente:

$$u = 2e + f, \quad 0 \leq f < 2$$

no puede haber 3 monedas de 2, ni 2 de 1 $\rightarrow$ $e = \left\lfloor \frac{u}{2} \right\rfloor$ y f es el resto. Este procedimiento es exactamente lo que realiza la función de selección elegida en el algoritmo Greedy.

¿Solución óptima? Otro sistema monetario

¿Porqué no funciona para el sistema de monedas {1c, 3c, 4c}?

Viendo el nuevo sistema monetario, podemos deducir fácilmente que en una solución óptima hay como máximo:

- 2 monedas de $1c$
- 3 monedas de $3c$

Poder llegar a tener 3 monedas de 3 implica que puede rentar más elegir monedas de valor 3 que de valor 4 →
No siempre es mejor usar una moneda de valor 4 →
La demostración anterior no sería válida.



UNIVERSIDAD DE GRANADA